

REDES DE DATOS - Teoría

Comisión 403 Cristian Medín

Resumen para segundo parcial: Capa de Transporte

Índice

Servicios y primitivas de transporte	2
Qué hace la capa de transporte	2
Tipos de servicio	2
Primitivas (sockets)	2
Características de transporte	3
Direccionamiento: puertos	3
Establecimiento de la conexión	3
Desconexión	3
Control de errores y de flujo	3
Multiplexación	3
Control de congestión en la capa 4	4
Caídas e inalámbricas	4
Protocolo TCP	4
Características	4
Encabezamiento TCP: 20 bytes fijos + hasta 40 de opciones	4
Establecimiento: 3-way handshake	5
Desconexión	5
Ventana deslizante y temporizadores	5
Control de congestión en TCP	6
SACK (Selective Acknowledgement)	6
Protocolo UDP	6
Encabezamiento y funcionamiento	6
Checksum y pseudoencabezado	6
RPC (llamada a procedimiento remoto)	7
Tiempo real: RTP y RTCP	7
TCP vs UDP	8
Preguntas de parciales	8
Parcial real de Medín (29/10/2024)	8
Otras preguntas de 2dos parciales (otros profesores, 2025)	9
Datos para memorizar	10
Fuentes	11

Servicios y primitivas de transporte

Qué hace la capa de transporte

La capa 4 le da a las aplicaciones un transporte **confiable, eficiente y de bajo costo**. Es la **primera capa que trabaja extremo a extremo**: las capas 1, 2 y 3 trabajan salto a salto, router por router, mientras que la 4 conecta directamente el host origen con el host destino. Por eso en la capa 4 el elemento principal es el **host**, como el router lo es en la capa 3.

La unidad de datos se llama **segmento** (antes, TPDU). Va dentro del paquete (capa 3), que a su vez va dentro de la trama (capa 2).

La **entidad de transporte** es el software y hardware que implementa la capa 4. Puede estar en el kernel del sistema operativo, en una librería, en la placa de red o en un proceso.

Responsabilidades (pregunta real de Medín):

- **Segmentar y reensamblar** los datos de la aplicación.
- **Entrega ordenada y confiable** (en TCP).
- **Control de errores y control de flujo**, extremo a extremo.
- **Multiplexación por puertos**: varias aplicaciones comparten una misma IP.
- **Control de congestión** (en TCP, junto con la capa 3).

Tipos de servicio

- **Orientado a conexión (TCP)**: tiene 3 fases (establecimiento, transferencia y liberación) y hace control de flujo.
- **Sin conexión (UDP)**: simple, sin controles. Manda y listo.

Primitivas (sockets)

Las operaciones que la capa 4 le ofrece a la aplicación se llaman **primitivas**. Las de Internet son los **sockets** de Berkeley (1983; en Windows, *winsock*).

Primitiva	Para qué
SOCKET	Crea un extremo de comunicación
BIND	Le asigna una dirección local, es decir, un puerto . En el cliente no hace falta
LISTEN	El servidor queda esperando conexiones (arma la cola)
ACCEPT	El servidor acepta un pedido de conexión
CONNECT	El cliente pide la conexión
SEND / RECEIVE	Mandar y recibir datos
CLOSE	Cierra; cada lado cierra su sentido (fin simétrico)

Orden del lado del servidor: SOCKET → BIND → LISTEN → ACCEPT. TCP ofrece un **flujo de bytes confiable** (*reliable byte stream*). Protocolos más nuevos: **SCTP** y SST.

Características de transporte

Direccionamiento: puertos

- La dirección de capa 4 es el **TSAP** (*Transport Service Access Point*), que en Internet es el **puerto: 16 bits**, o sea 65.536 puertos por IP. Con una sola IP, un host atiende muchas aplicaciones.
- **Puertos conocidos:** FTP **20/21**, Telnet **23**, SMTP **25**, HTTP **80**. Los menores a 1024 son reservados.
- Si no se conoce el puerto del servicio: un **port mapper** (puerto 111) traduce servicio a puerto, o un **server process** escucha varios puertos en nombre de otros.

Establecimiento de la conexión

- El cliente manda un pedido de conexión (*connection request*) y espera la aceptación.
- **Problema principal: los segmentos duplicados.** Un segmento demorado que llega tarde puede repetir una operación. Por ejemplo, una transferencia bancaria hecha dos veces.
- **Solución:** limitar la vida de los segmentos (en Internet, **T ≈ 120 s**) y **no reusar un número de secuencia** antes de ese tiempo.
- **3-way handshake** (Tomlinson, 1975): cada lado elige su número de secuencia inicial y el otro lo confirma. En TCP el número de secuencia es de **32 bits** y el inicial es **aleatorio**, para que no se pueda predecir.

Desconexión

- **Asimétrica:** uno corta y se corta todo, como el teléfono. Se pueden perder datos.
- **Simétrica:** cada sentido se cierra por separado, como dos canales independientes. Es la de TCP.
- **Problema de los dos ejércitos:** nunca hay forma de asegurar que el **último mensaje** llegó, porque confirmarlo requiere otro mensaje. Por eso la desconexión también es un intercambio de 3 vías, y cada lado **corta por su cuenta con un timeout** si los mensajes se pierden.

Control de errores y de flujo

- Son los mismos mecanismos que en la capa 2, pero **extremo a extremo**. La capa 4 detecta errores que la capa 2 no ve, por ejemplo los que se producen dentro de un router.
- **ARQ** (*Automatic Repeat reQuest*) = ventana deslizante + número de secuencia + retransmisión por timeout.
- **Checksum:** obligatorio en TCP, opcional en UDP.
- **Buffers:** pueden ser de tamaño fijo, de tamaño variable o **circulares**, que son los mejores. Cada ACK informa qué llegó y **cuánto espacio libre** queda. La ventana es **dinámica**: se ajusta a la memoria del receptor y a la capacidad de la red.

Multiplexación

- **Multiplexación:** varias conexiones (Zoom, mail, WhatsApp) comparten el mismo enlace y la misma IP; los **puertos** las separan.

- **Multiplexación inversa:** una conexión usa **varios caminos o conexiones en paralelo** para ganar velocidad. La hace **SCTP**. **TCP no** la hace: es unicast, una conexión entre dos extremos.

Control de congestión en la capa 4

- La congestión **ocurre en los routers** (capa 3), pero **la causa el tráfico** que generan los hosts (capa 4). Por eso es una responsabilidad **conjunta**. El objetivo es que cada fuente tenga una buena tasa de bits sin saturar la red.
- **Tasa justa (*max-min fair*):** no se puede subir la tasa de una fuente sin bajar la de otra que tenga igual o menos. Ejemplo con 4 fuentes: B, C y D reciben 1/3 y A recibe 2/3.
- **AIMD** (*Additive Increase, Multiplicative Decrease*; Chiu y Jain, 1989): **sube de a poco** (suma) y, ante congestión, **baja a la mitad** (multiplica por 1/2). Converge a un reparto eficiente y justo. Es la ley de TCP. No es perfectamente justa: favorece a las conexiones con RTT corto. Cualquier protocolo nuevo tiene que ser "**TCP-friendly**".
- **Variantes modernas:** **CUBIC** (Linux, reacciona a pérdidas), **Compound** (Windows, pérdidas y retardo), **FAST TCP** (retardo).

Caídas e inalámbricas

- **Si el receptor se cae** (*crash*), el emisor tiene 4 estrategias posibles: retransmitir siempre, nunca, o según el estado en que haya quedado. **Ninguna es transparente** para las capas superiores.
- **WiFi:** pierde cerca del **10 %** de las tramas, mientras que TCP tolera alrededor del 1 %. Por eso se retransmite en la **capa 2** (stop-and-wait), en milisegundos, sin que la capa 4 se entere.
- **Satélite:** los tiempos de la capa 2 se parecen a los de la capa 4, así que se usa **FEC** (corrección hacia adelante) en vez de retransmitir.

Protocolo TCP

Características

- **TCP** (*Transmission Control Protocol*, RFC 793, 1981): conexión **confiable extremo a extremo** sobre una red que no lo es.
- **Full-duplex, unicast, punto a punto. No soporta multicast ni broadcast.**
- **Flujo de bytes** (*byte stream*): el receptor no sabe en qué pedazos se mandaron los bytes.
- El segmento puede llegar a 64 KB, pero en la práctica es de ≤ 1460 B de datos, para que con los headers TCP e IP entre en una **trama Ethernet de 1500 B** sin fragmentar.
- Los procesos que atienden servicios en segundo plano se llaman **daemons** (por ejemplo, el daemon de FTP en los puertos 20/21).

Encabezamiento TCP: 20 bytes fijos + hasta 40 de opciones

- **Puerto origen y puerto destino** (16 bits cada uno).
- **Número de secuencia y número de acknowledgement** (32 bits cada uno).
- **Largo del header** (4 bits): indica dónde empiezan los datos.
- **Flags:**

Flag	Significado
SYN	Sincronizar: abrir la conexión
ACK	El segmento trae un acknowledgement válido
FIN	El emisor no tiene más datos: cerrar su sentido
RST	Reset: reiniciar la conexión
PSH	Entregar los datos a la aplicación ya, sin esperar a llenar el buffer
URG	Hay datos urgentes ; el puntero urgente indica dónde terminan
ECE / CWR	Congestión (ECN): ECE pide bajar la velocidad, CWR confirma que se bajó

- **Tamaño de ventana** (16 bits): cuántos bytes se pueden mandar sin ACK. Es el **control de flujo**; el máximo es $2^{16} = 64$ KB.
- **Checksum: obligatorio**. Cubre el segmento más un **pseudoencabezado IP**.
- **Opciones:**
 - **MSS** (tamaño máximo de segmento): por defecto, **556 B totales**, que todo host tiene que aceptar (536 de datos + 20 de header TCP).
 - **Escala de ventana**: agranda la ventana hasta $2^{30} = 1$ GB, para enlaces rápidos o largos.
 - **Timestamp**.
 - **SACK**.

Datos urgentes: con **URG = 1**, el **puntero urgente** marca hasta dónde llegan los datos urgentes dentro del segmento, para que el receptor los procese antes que el resto del buffer. Ejemplo: un Ctrl+C para cortar un proceso remoto.

Establecimiento: 3-way handshake

Cliente	Servidor (en LISTEN)
---- SYN=1, ACK=0, seq=x ----->	CONNECT: pide conexión (+ MSS)
<--- SYN=1, ACK=1, seq=y, ack=x+1 -	el servidor acepta
---- ACK=1, ack=y+1 ----->	el cliente confirma
conexión establecida	

Desconexión

Se cierra **cada sentido por separado**, con un **FIN** y su **ACK** en cada dirección: 4 segmentos en total. Si alguno se pierde, un **timeout** completa el cierre. Es la salida práctica al problema de los dos ejércitos.

Ventana deslizante y temporizadores

- **Control de flujo con ventana:** el receptor anuncia cuánto espacio libre le queda. Ejemplo: con un buffer de 4 KB y segmentos de 2 KB, después de 2 segmentos el buffer se llena y el emisor espera hasta que un ACK anuncie espacio.
- **Algoritmo de Nagle:** junta datos chicos para mandar segmentos más grandes y no desperdiciar headers.
- **Temporizador de retransmisión:** se **ajusta dinámicamente** según el RTT medido. Ronda el segundo en la capa 4, unas 1000 veces más que en la capa 2. Si es muy grande, la conexión se demora; si es muy chico, retransmite de más.

Control de congestión en TCP

- La **ventana de congestión** limita los bytes en tránsito sin ACK. TCP manda el mínimo entre la ventana de congestión y la ventana del receptor.
- **Reloj de ACKs**: cada ACK que vuelve habilita a mandar más, así los ACK marcan el ritmo de envío.
- **Slow start**: arranca con una ventana chica y la **duplica en cada RTT**, porque crece un segmento por cada ACK. Así sube rápido hasta encontrar el límite.
- **Incremento aditivo**: pasado el umbral, crece de a un segmento por RTT.
- **Ante una pérdida**: la ventana **baja a la mitad** (AIMD).

Versión	Qué hace ante pérdida
TCP Tahoe (1988)	Slow start + incremento aditivo. Ante 3 ACK duplicados asume pérdida: fija el umbral en la mitad y vuelve a empezar con slow start
TCP Reno (1990)	Agrega fast recovery : ante 3 ACK duplicados sigue desde la mitad de la ventana, sin volver a slow start

La diapositiva de Medín dice que Tahoe "comienza nuevamente con una ventana igual a la mitad de la anterior". Más preciso (Tanenbaum): lo que se fija en la mitad es el **umbral**, y la ventana vuelve a arrancar desde 1 segmento con slow start hasta alcanzarlo. Reno, en cambio, retoma directamente desde la mitad. Las versiones actuales (CUBIC en Linux, Compound en Windows) se parecen a Reno.

SACK (Selective Acknowledgement)

El receptor informa **rangos de lo que sí recibió**, así el emisor sabe exactamente qué retransmitir. Ejemplo: si se pierden el 2 y el 5, manda el ACK del 1 más un SACK de 3, 4 y 6.

Protocolo UDP

Encabezamiento y funcionamiento

- **UDP** (*User Datagram Protocol*, RFC 768): **sin conexión**. Manda segmentos de puerto a puerto y deja todo lo demás a la aplicación.
- **Header de 8 bytes**: puerto origen (2), puerto destino (2), longitud (2) y checksum (2).
- El segmento mide de **8 B a 65.515 B** (65.535 menos los 20 del header IP).
- **No hace** control de flujo, control de congestión ni retransmisiones. No establece ni libera conexión. Lo único que hace es **separar comunicaciones por puerto**.
- **Control de congestión** (pregunta real de Medín): **UDP no la controla**. Envía al ritmo de la aplicación y no se adapta al estado de la red, así que puede saturarla. Si hace falta control, lo implementa la aplicación.

Checksum y pseudoencabezado

- **El checksum es opcional**: si no se usa, va en ceros. Conviene desactivarlo solo si los datos no son críticos, como en VoIP.
- Cubre el segmento más un **pseudoencabezado IP**: IP origen, IP destino, un byte en cero, el **protocolo (17)** y la longitud. Es una suma en complemento a uno de palabras de 16 bits.

- Que la capa 4 controle campos de la capa 3 **viola la estructura de capas**, pero permite detectar paquetes que llegaron al host equivocado.
- **Ojo, las fuentes no coinciden sobre qué hace UDP ante un error.** La diapositiva de la cátedra dice que **avisa a las capas superiores y deja que ellas decidan**; el documento de preguntas de Medín dice que **descarta** el datagrama. Lo seguro: **UDP detecta el error con el checksum, pero no lo corrige ni retransmite.** Confirmar con Medín qué respuesta espera.

RPC (llamada a procedimiento remoto)

Un host ejecuta un proceso en otro como si fuera una función local; por ejemplo, al pedir una canción en Spotify.

1. El cliente ejecuta el **client stub**.
2. El stub empaqueta los parámetros (**marshaling**).
3. El sistema operativo del cliente manda el mensaje.
4. El sistema operativo del servidor se lo pasa al **server stub**.
5. El server stub llama al servidor.

La respuesta vuelve por el camino inverso. Es distinto de los sockets, y **UDP sirve para transportarlo**.

Tiempo real: RTP y RTCP

- El **streaming** (radio, VoIP, videoconferencia, música, películas) va sobre **UDP + RTP** (*Real-Time Protocol*, RFC 3550).
- **RTP** encapsula los datos de tiempo real en segmentos UDP. **No retransmite ni confirma**, porque un paquete retransmitido llegaría tarde para reproducirse.
 - Define **perfiles de codificación**: MP3, GSM, PCM y otros.
 - Cuida el **sincronismo**: con el **timestamp**, cada muestra se reproduce en el momento en que se grabó, y se sincronizan varias cadenas, como el video y el audio en distintos idiomas.
 - **Header RTP**: versión, P (relleno), X (extensión), CC (cantidad de fuentes), M (marca), **tipo de carga** (la codificación), **número de secuencia**, **timestamp** e identificadores de **fuentes de sincronización** y de **fuentes contribuyentes**.
- **RTCP** (*Real-time Transport Control Protocol*) es el **control** de RTP: da **sincronismo y realimentación**. Con esa información se **baja la velocidad y la calidad** cuando la red anda mal, y al revés.
- **Jitter y buffer**: el **jitter** es la variación del retardo entre paquetes. El receptor guarda los datos en un **buffer** y fija un punto de reproducción que cubra cerca del 99 % de las muestras:
 - unos **10 s** en streaming de una vía (Netflix, Spotify);
 - **mucho menos** en una videoconferencia (Zoom), para poder conversar.

TCP vs UDP

	TCP	UDP
Conexión	Orientado a conexión (3-way handshake)	Sin conexión: manda directamente
Confiabilidad	Garantiza entrega y orden; retransmite	No garantiza entrega ni orden
Ante un error	Retransmite hasta recibir el ACK	No retransmite (descarta o avisa a la aplicación)
Control de flujo y congestión	Sí (ventana, AIMD, slow start)	No
Checksum	Obligatorio	Opcional
Header	20 B + opciones	8 B
Datos	Flujo de bytes	Mensajes individuales
Destinos	Solo unicast	Unicast, broadcast y multicast
Usos	Web, correo, FTP, transferencia bancaria	Voz y video en tiempo real (streaming , con RTP), DNS, DHCP

Preguntas de parciales

Parcial real de Medín (29/10/2024)

1) ¿Qué responsabilidades tiene la capa de transporte?

Darle a las aplicaciones una comunicación **extremo a extremo** confiable y eficiente entre hosts. Para eso:

- **segmenta** los datos y los **reensambla** en el destino;
- garantiza **entrega ordenada y confiable** (TCP);
- hace **control de errores** y **control de flujo**;
- **multiplexa por puertos**, para que varias aplicaciones compartan una IP;
- hace **control de congestión** (TCP).

2) ¿Cómo controla la congestión el protocolo UDP?

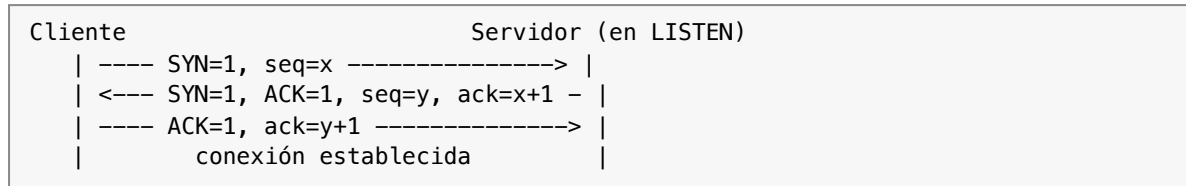
No la controla. UDP no tiene control de congestión ni de flujo: envía al ritmo de la aplicación, sin adaptarse a la red, y por eso puede saturarla. Si hace falta, el control lo implementa la propia aplicación (por ejemplo, RTCP baja la calidad del streaming).

3) ¿Qué sucede si UDP detecta un error? ¿Y TCP?

- **UDP** detecta el error con el checksum (si está activado), pero **no retransmite ni corrige**. Según el documento de preguntas de Medín, **descarta** el datagrama. Según la diapositiva de la cátedra, **avisa a las capas superiores** para que decidan. En cualquier caso, la recuperación queda en manos de la aplicación.
- **TCP** detecta el error con el checksum (obligatorio), descarta el segmento dañado y **lo retransmite**: el emisor no recibe el ACK y reenvía cuando vence el temporizador. Mantiene

el orden y la confiabilidad.

4) Grafique un ejemplo de conexión 3-way handshake.



El cliente pide la conexión con SYN y su número de secuencia inicial; el servidor acepta con SYN+ACK, confirma el número del cliente y propone el suyo; el cliente confirma con ACK. Desde ahí, los datos van en los dos sentidos.

5) Enuncie diferencias entre TCP y UDP.

TCP es orientado a conexión, confiable (retransmite y ordena), con control de flujo y de congestión, checksum obligatorio, header de 20 B o más y flujo de bytes; es solo unicast. UDP es sin conexión, no garantiza entrega ni orden, no controla flujo ni congestión, tiene checksum opcional, header de 8 B y manda mensajes individuales; admite broadcast y multicast. TCP se usa en web, correo y transferencias; UDP, en streaming, VoIP, DNS y DHCP.

Otras preguntas de 2dos parciales (otros profesores, 2025)

6) Explique la multiplexación que realiza la capa de transporte al ejecutar varias aplicaciones a la vez.

Varias aplicaciones de un mismo host (navegador, mail, Zoom) comparten **la misma IP y el mismo enlace**. La capa 4 distingue cada conversación por el **número de puerto**: al enviar, marca cada segmento con el puerto de la aplicación; al recibir, mira el puerto destino y le entrega los datos a la aplicación que corresponde.

7) ¿Qué es la multiplexación inversa y qué protocolo la realiza?

Es lo contrario: **una sola conexión usa varios caminos o conexiones en paralelo** para sumar ancho de banda o tolerar fallas. La hace **SCTP**. TCP no, porque es una única conexión unicast entre dos extremos.

8) ¿A qué se denomina "socket"?

Es la **interfaz de primitivas** que la capa 4 le ofrece a la aplicación (SOCKET, BIND, LISTEN, ACCEPT, CONNECT, SEND, RECEIVE, CLOSE) y, a la vez, el **extremo de una conexión**, identificado por IP + puerto. Una conexión TCP queda identificada por los dos sockets de sus extremos.

9) ¿Cuál es el mecanismo de TCP para enviar datos urgentes?

Se activa el flag **URG** y el **puntero urgente** indica dónde terminan los datos urgentes dentro del segmento. El receptor los atiende antes que los datos que están esperando en el buffer. Ejemplo: un Ctrl+C para interrumpir un proceso remoto.

10) Explique el proceso ARQ y el control de errores de TCP.

ARQ (*Automatic Repeat reQuest*) es retransmitir automáticamente lo que no se confirma. TCP numera cada byte (número de secuencia), el receptor confirma con **ACK acumulativos** (o

SACK para rangos), y el emisor retransmite cuando **vence el temporizador** o cuando recibe **3 ACK duplicados**. El checksum obligatorio detecta los segmentos dañados, que se descartan y terminan retransmitidos.

11) ¿Qué es la tasa justa en TCP y cuál es la ley óptima para controlar la congestión?

Tasa justa (max-min): repartir el ancho de banda de modo que no se pueda subir la tasa de una fuente sin bajar la de otra que tenga igual o menos. La **ley óptima es AIMD**: aumentar la tasa de a poco (aditivamente) y, ante congestión, bajarla a la mitad (multiplicativamente). Así las fuentes convergen a un reparto eficiente y justo.

12) Explique slow start, incremento aditivo, TCP Tahoe y TCP Reno.

- **Slow start:** la ventana de congestión arranca chica y se **duplica en cada RTT** hasta llegar al umbral o a una pérdida.
- **Incremento aditivo:** pasado el umbral, crece **un segmento por RTT**.
- **Tahoe:** ante una pérdida (3 ACK duplicados o timeout), fija el umbral en la mitad y **vuelve a slow start** desde cero.
- **Reno:** agrega **fast recovery**: ante 3 ACK duplicados, **sigue desde la mitad** de la ventana sin volver a slow start.

13) ¿Qué protocolo de capa 4 conviene para streaming y cuál para una transferencia bancaria?

- **Streaming: UDP (con RTP).** Importa que llegue a tiempo, no que llegue todo: retransmitir un paquete de video viejo no sirve.
- **Transferencia bancaria: TCP.** No puede perderse, duplicarse ni desordenarse ningún dato.

Datos para memorizar

- La capa 4 es **extremo a extremo**; la unidad es el **segmento**; la dirección es el **puerto** (16 bits).
- Puertos: **FTP 20/21 · Telnet 23 · SMTP 25 · HTTP 80 · DHCP 67/68 · port mapper 111**.
- Header **TCP: 20 B + hasta 40 de opciones**; header **UDP: 8 B**. UDP es el protocolo **17** y TCP el **6**.
- Checksum: **obligatorio en TCP, opcional en UDP**; los dos usan **pseudoencabezado IP**.
- Ventana TCP: máximo **64 KB** (2^{16}); con escala de ventana, hasta **1 GB** (2^{30}). MSS por defecto: **556 B** totales.
- Vida máxima de un segmento: **T ≈ 120 s**. Número de secuencia: **32 bits**, inicial aleatorio.
- Flags: **SYN** abrir · **ACK** confirma · **FIN** cerrar · **RST** reiniciar · **PSH** entregar ya · **URG** urgente · **ECE/CWR** congestión.
- **AIMD**: sube sumando, baja a la mitad. **Tahoe** vuelve a slow start; **Reno** sigue desde la mitad (fast recovery).
- **SCTP** hace multiplexación inversa; **TCP** no (solo unicast).
- Streaming: **UDP + RTP/RTCP**. WiFi retransmite en **capa 2**; satélite usa **FEC**.

Fuentes

- Teóricos de la cátedra (Medín): 1 - *Servicios y primitivas de transporte*, 2 - *Características de transporte*, 3 - *Protocolo TCP*, 4 - *Protocolo UDP*.
- *Preguntas y respuestas de parciales de Medín* y el 2do parcial real del 29/10/2024 (com. 403).
- Preguntas de 2dos parciales de 2025 de otros profesores de la cátedra (Travaglino, Pastori), como práctica.
- Tanenbaum y Wetherall, *Redes de Computadoras*, 5ª ed., cap. 6.
- **El temario del 2do teórico de 2026 (27/10) todavía no está confirmado.** Este resumen asume que es Capa de Transporte, como en 2024.